

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ
FACULTAD DE CIENCIAS E INGENIERÍA

ALGORITMIA Y ESTRUCTURA DE DATOS

5ta. práctica (tipo B)
(Segundo Semestre 2025)

Duración: 1h 50 min.

- **No puede utilizar apuntes, solo hojas sueltas en blanco.**
- En cada función el alumno deberá incluir, a modo de comentario, la forma de solución que utiliza para resolver el problema. De no incluirse dicho comentario, el alumno perderá el derecho a reclamo en esa pregunta.
- No puede emplear plantillas o funciones no vistas en los cursos de programación de la especialidad.
- Los programas deben ser desarrollados en el lenguaje C++. Si la implementación es diferente a la estrategia indicada o no la incluye, la pregunta no será corregida.
- Un programa que no muestre resultados coherentes y/o útiles será corregido sobre el 50% del puntaje asignado a dicha pregunta.
- Debe utilizar comentarios para explicar la lógica seguida en el programa elaborado. El orden será parte de la evaluación.
- **Solo está permitido acceder a la plataforma de PAIDEIA, cualquier tipo de navegación, búsqueda o uso de herramientas de comunicación se considera plagio por tal motivo se anulará la evaluación y se procederá con las medidas disciplinarias dispuestas por la FCI.**
- Para esta evaluación solo se permite el uso de las librerías `iostream`, `iomanip`, `limits`, `cstring`, `cmath` o `fstream`
- Su trabajo deberá ser subido a PAIDEIA.
- **Es obligatorio usar como compilador CLion.**
- Los archivos deben llevar como nombre su código de la siguiente forma `codigo_LAB5_P#&` (donde # representa el número de la pregunta a resolver y & la alternativa)

Pregunta 1 (10 puntos)

Inserción y balanceo en un ABB tipo AVL (rotaciones)

Una empresa almacena los DNI de sus clientes en un **Árbol Binario de Búsqueda auto-balanceado** (tipo AVL), para garantizar que las operaciones de búsqueda e inserción se mantengan en tiempo cercano a $O(\log n)$.

Recordemos que en un ABB:

- Todas las claves del subárbol izquierdo de un nodo **N** son menores que la clave de **N**.
- Todas las claves del subárbol derecho de un nodo **N** son mayores que la clave de **N**.
- En un Árbol Binario de Búsqueda, las inserciones siguen únicamente la regla de ordenamiento (izquierda < nodo < derecha), por lo que, dependiendo del orden en que llegan los datos, el árbol puede volverse progresivamente “inclinado” hacia un lado. Cuando una rama crece mucho más que la otra, la **altura del árbol aumenta** de manera desproporcionada y su **factor de balance (FB)** se sale del rango permitido (-1, 0, +1). Esto se conoce como **desbalanceo** y tiene un impacto directo en el rendimiento: las operaciones de búsqueda, inserción y recorrido

dejan de ser $O(\log n)$ y pueden degradarse hasta $O(n)$, como si el árbol fuera prácticamente una lista enlazada.

En un árbol **AVL**, además:

- Para cada nodo, se define el **factor de balance** como:
 $FB(nodo) = altura(subárbol_izq) - altura(subárbol_der)$
- El árbol está balanceado si para **todo** nodo se cumple: $FB(nodo) \in \{-1, 0, +1\}$.
- Después de cada inserción es posible que el árbol se desbalancee, y se deteriore.
- Para evitar este deterioro, los árboles AVL aplican **rotaciones** inmediatamente después de detectar un desbalance. Una rotación es una operación local que **reorganiza** un pequeño subárbol, redistribuyendo sus nodos de forma que se recupere el balance sin alterar el orden lógico del ABB. Gracias a estas rotaciones (LL, RR, LR y RL), el árbol mantiene su altura lo más baja posible y garantiza un rendimiento óptimo incluso en el peor caso.

2. Pseudocódigo y explicación del balanceo (AVL) (3ptos)

2.1. Estructura del nodo

```
struct Nodo {  
    int dni;  
    int altura;  
    Nodo *izq;  
    Nodo *der;  
};
```

```
funcion ALTURA(nodo):  
    si nodo es NULO entonces  
        retornar 0  
    fin si  
    retornar nodo.altura  
fin funcion
```

```
funcion FACTOR_BALANCE(nodo):  
    si nodo es NULO entonces  
        retornar 0  
    fin si  
  
    altura_izq  $\leftarrow$  ALTURA(nodo.izq)  
    altura_der  $\leftarrow$  ALTURA(nodo.der)  
  
    retornar altura_izq - altura_der  
fin funcion
```

2.2. Rotaciones:

Rotación simple a la derecha (Right Rotate)

Caso típico **LL**: el subárbol izquierdo está “pesado”.

Pseudocódigo:

```
funcion rotacionDerecha(y):  
    x = y.izq  
    T2 = x.der  
  
    // rotar  
    x.der = y  
    y.izq = T2  
  
    // actualizar alturas  
    y.altura = 1 + max(altura(y.izq), altura(y.der))  
    x.altura = 1 + max(altura(x.izq), altura(x.der))  
  
    retornar x // nueva raíz del subárbol  
fin funcion
```

Rotación simple a la izquierda (Left Rotate)

Caso típico **RR**: el subárbol derecho está “pesado”.

```
funcion rotacionIzquierda(x):  
    y = x.der  
    T2 = y.izq  
  
    // rotar  
    y.izq = x  
    x.der = T2  
  
    // actualizar alturas  
    x.altura = 1 + max(altura(x.izq), altura(x.der))  
    y.altura = 1 + max(altura(y.izq), altura(y.der))  
  
    retornar y // nueva raíz del subárbol  
fin funcion
```

Casos dobles (LR y RL)

- **Caso Izquierda-Derecha (LR):**
 1. Rotar a la **izquierda** el hijo izquierdo.
 2. Rotar a la **derecha** el nodo actual.

```
si FB(nodo) > 1 y clave > nodo.izq.dni:  
    nodo.izq = rotacionIzquierda(nodo.izq)  
    retornar rotacionDerecha(nodo)
```

- **Caso Derecha-Izquierda (RL):**
 1. Rotar a la **derecha** el hijo derecho.
 2. Rotar a la **izquierda** el nodo actual.

si **FB**(nodo) < -1 y clave < nodo.der.dni:
 nodo.der = rotacionDerecha(nodo.der)
 retornar **rotacionIzquierda**(nodo)

2.3. Pseudocódigo completo de **insertarAVL**

```
funcion insertarAVL(nodo, clave) retorna nodo:
  si nodo == NULL entonces
    nodo = <COMPLETAR>
    nodo.dni = <COMPLETAR>
    nodo.izq = <COMPLETAR>
    nodo.der = <COMPLETAR>
    nodo.altura = <COMPLETAR>
    retornar nodo
  fin si

  // Inserción ABB normal
  si clave < nodo.dni entonces
    nodo.izq = <COMPLETAR>
  sino si clave > nodo.dni entonces
    nodo.der = <COMPLETAR>
  sino
    // clave igual, no insertamos duplicados
    retornar nodo
  fin si

  // 1) Actualizar altura del nodo actual
  nodo.altura = 1 + max( altura(nodo.izq), altura(nodo.der) )

  // 2) Calcular factor de balance
  FB = <COMPLETAR>

  // 3) Revisar los 4 casos

  // Caso LL
  <COMPLETAR>

  // Caso RR
  <COMPLETAR>

  // Caso LR
  <COMPLETAR>

  // Caso RL
  <COMPLETAR>

  retornar nodo
fin funcion
```

3. Implementación (7ptos)

Utilizando el pseudocódigo desarrollado en las secciones anteriores para la inserción y el balanceo de un Árbol AVL:

a) Implemente en C++ las funciones necesarias para construir un **Árbol AVL** que almacene valores enteros (DNIs).

b) Declare en el `main()` un **arreglo de DNIs** con valores de prueba, por ejemplo:

```
{72649318, 50823147, 81234567, 40987654, 65012345, 94561237, 30124598};
```

c) Inserte cada uno de los DNIs en el árbol AVL utilizando la función `insertarAVL` implementada a partir del pseudocódigo.

d) Implemente una función de **recorrido inorden** (inorder traversal) y muestre en pantalla los DNIs resultantes en dicho orden.

Nota: Para obtener el puntaje completo, la implementación debe respetar fielmente la lógica del pseudocódigo de AVL

Salida esperada (formato orientativo):

Insertando valores en el AVL:

Insertar 72649318

Insertar 50823147

Insertar 81234567

Insertar 40987654

Insertar 65012345

Insertar 94561237

Insertar 30124598

Recorrido inorden del AVL (clave y altura):

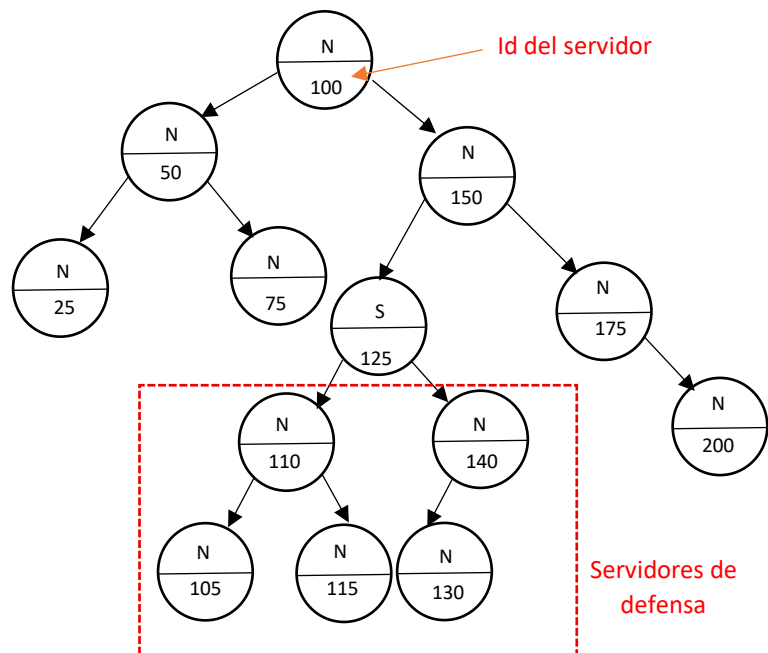
30124598(h:1) 40987654(h:2) 50823147(h:3) 65012345(h:1) 72649318(h:4) 81234567(h:2)
94561237(h:1)

Pregunta 2 (10 puntos)

Luego de los reiterados intentos fallidos del equipo de especialistas en algoritmia para detener el despliegue de SkyNerd, existe una última esperanza para poder eliminar a este sistema inteligente de la red. Es así como un reducido grupo de programadores que ha sobrevivido logra obtener la siguiente información:

- El sistema inteligente ha formado una red basada en un árbol binario de búsqueda, para su defensa
- Se sabe que cada nodo del árbol guarda un flag si es SkyNerd (S) o no (N), además de un número id del servidor el cual sirve para el ordenamiento del ABB
- El servidor inteligente cuenta con una defensa formada por los servidores hijos de este subárbol
- Para vencer a este servidor debe seguir los siguientes pasos:
 - Ubicar dentro del ABB al servidor nocivo, pero sin pasar por ningún servidor hijo de este.
 - Una vez ubicado SkyNerd se debe eliminar la red de servidores hijos que lo defienden, pero la única forma de quitar estos servidores es primero dejarlo sin hijos y luego recién proceder con la eliminación del nodo. Este proceso se debe a que al igual que SkyNerd los servidores hijos de cada nodo sirven de defensa del servidor padre.

A continuación, un ejemplo:



La salida para este ejemplo para ubicar a SkyNerd será:

N-100 N-50 N-25 N-75 N-150 S-125

La salida del subárbol a eliminar es:

N-105 N-115 N-110 N-130 N-140

Desarrollar los algoritmos necesarios dejar sin defensa al servidor nocivo, de tal forma que primero puedan ubicar a SkyNerd bajos las condiciones indicadas, y luego eliminar a todo el subárbol formado por sus hijos. Todos los desarrollos realizados para la solución deben ser iterativos. Para el borrado de los hijos no debe usar la función **eliminar**.

Al finalizar el laboratorio, comprima la carpeta de su proyecto empleando el programa Zip que viene por defecto en el Windows, **no se aceptarán los trabajos compactados con otros programas como RAR, WinRAR, 7zip o similares**. Luego súbalo a la tarea programa en Paideia para este laboratorio.

Profesores del curso:

Ana Roncal
Fernando Huamán
David Allasi
Rony Cueva
Erasmó Gómez

San Miguel, 29 de noviembre del 2025